OPEN

Compute Project

Universal Serial Bus
Device Class Specification for
OCP Open Boot and Management Framework
Interface Consolidation Protocol

Version 1.0.0
March 4th, 2026

Editor:
Mariusz Oriol — NVIDIA Corporation
Contributors:
Janusz Jurski — Intel Corporation
Bharat Pillilli — Microsoft Corporation

Table of Contents

1. LICENSE 3

1.1 OPEN WEB FOUNDATION (OWF) CLA 3

2. INTRODUCTION 5

2.1 REVISION HISTORY 5
2.2 SYNTAX AND CONVENTIONS 5
2.3 RELATED DOCUMENTS 5
2.4 TERMS AND ABBREVIATIONS 5

3. OVERVIEW 6

4. OCP OBMF-ICP USB INTERFACE DEFINITION 7

4.1 USB INTERFACE DESCRIPTOR 7
4.2 OBMF-ICP CLASS-SPECIFIC FUNCTIONAL DESCRIPTOR 8
4.3 USB INTERFACE ENDPOINT DESCRIPTORS 9
4.3.1 USB Interface Endpoint Descriptor for Bulk OUT (Mandatory) 9
4.3.2 USB Interface Endpoint Descriptor for Bulk IN (Mandatory) 9
4.3.3 USB Interface Endpoint Descriptor for Interrupt OUT (Optional) 9
4.3.4 USB Interface Endpoint Descriptor for Interrupt IN (Optional) 10
4.4 USB DESCRIPTORS FOR OCP OBMF-ICP INTERFACE 10

5. OCP OBMF-ICP OVER USB DEVICE CLASS 11

5.1 LENGTH OF THE OBMF-ICP MESSAGE THAT SPANS ACROSS MULTIPLE PACKETS: 11

6. COMPATIBILITY 13

7. APPENDIX: USB INTERFACE IMPLEMENTATION NOTES (*INFORMATIVE ONLY*) 14

7.1 USB TRANSFER ERROR HANDLING 14
7.2 USB INTERFACE ERROR RECOVERY 14

8. ABOUT OPEN COMPUTE FOUNDATION 16

1. License

THE UPDATED DEFAULT CONTRIBUTOR LICENSE AGREEMENT (CLA) IS OWFa 0.9. PLEASE VERIFY THE CORRECT CLA/FSA IS USED AND EXECUTED FOR THIS CONTRIBUTION.

1.1 Open Web Foundation (OWF) CLA

Contributions to this Specification are made under the terms and conditions set forth in Modified Open Web Foundation Agreement 0.9 (OWFa 0.9). (As of October 16, 2024) ("Contribution License") by:

- Advanced Micro Devices
- Arm Limited
- ASPEED Technology, Inc.
- Dell Technologies
- Google Inc.
- Intel Corporation
- Microsoft Corporation
- Nuvoton Technology Israel Ltd.
- NVIDIA

Usage of this Specification is governed by the terms and conditions set forth in Modified OWFa 0.9 Final Specification Agreement (FSA) (As of October 16, 2024) ("Specification License").

You can review the applicable Specification License(s) referenced above by the contributors to this Specification on the OCP website at https://www.opencompute.org/contributions/templates-agreements.

For actual executed copies of either agreement, please contact OCP directly.

Notes:

The above license does not apply to the Appendix or Appendices. The information in the Appendix or Appendices is for reference only and non-normative in nature.

NOTWITHSTANDING THE FOREGOING LICENSES, THIS SPECIFICATION IS PROVIDED BY OCP "AS IS" AND OCP EXPRESSLY DISCLAIMS ANY WARRANTIES (EXPRESS, IMPLIED, OR OTHERWISE), INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, OR TITLE, RELATED TO THE SPECIFICATION. NOTICE IS HEREBY GIVEN, THAT OTHER RIGHTS NOT GRANTED AS SET FORTH ABOVE, INCLUDING WITHOUT LIMITATION, RIGHTS OF THIRD PARTIES WHO DID NOT EXECUTE THE ABOVE LICENSES, MAY BE IMPLICATED BY THE IMPLEMENTATION OF OR COMPLIANCE WITH THIS SPECIFICATION. OCP IS NOT RESPONSIBLE FOR IDENTIFYING RIGHTS FOR WHICH A LICENSE MAY BE REQUIRED IN ORDER TO IMPLEMENT THIS SPECIFICATION. THE ENTIRE RISK AS TO OCP Open Boot and Management Framework – Interface Consolidation Protocol IMPLEMENTING OR OTHERWISE USING THE SPECIFICATION IS ASSUMED BY YOU. IN NO EVENT WILL OCP BE LIABLE TO YOU FOR ANY MONETARY

DAMAGES WITH RESPECT TO ANY CLAIMS RELATED TO, OR ARISING OUT OF YOUR USE OF THIS SPECIFICATION, INCLUDING BUT NOT LIMITED TO ANY LIABILITY FOR LOST PROFITS OR ANY CONSEQUENTIAL, INCIDENTAL, INDIRECT, SPECIAL OR PUNITIVE DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS SPECIFICATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND EVEN IF OCP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

2. Introduction

This document describes proposed requirements and specifications for Universal Serial Bus (USB) devices that support the Open Boot and Management Framework (OBMF) Interface Consolidation Protocol (OBMF-ICP) capability as defined by Opencompute.org. This document outlines the native Universal Serial Bus Device Class Specification for OBMF-ICP protocol.

2.1 Revision History

Revision	Date	Guiding Contributor(s)	Description
1.0.0	02-05-2026	Mariusz Oriol, NVIDIA	Final Release

2.2 Syntax and conventions

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

2.3 Related Documents

For details to the OBMF-ICP protocol please refer to published version of OBMF-ICP specification at https://www.opencompute.org/contributions

USB Implementers Forum. Universal Serial Bus Specification, Revision 2.0, April 27th, 2000 https://www.usb.org/document-library/usb-20-specification

2.4 Terms and Abbreviations

The meanings of some words have been stretched to suit the purposes of this document. These definitions are intended to clarify the discussions that follow.

OBMF	Open Boot and Management Framework as defined by opencompute.org
OBMF-ICP	OCP OBMF Interface Consolidation Protocol
OBMF-ICP Message	Data sent to specific OBMF-ICP Channel that fits into USB Transfer.
Packet	Atomic unit transmitted on USB wire (Token, Data, Handshake). The terms "USB Packet" and "packet" are used interchangeably per USB 2.0 Specification.
Transfer	Logical operation comprising one or more packets (Control, Bulk or Interrupt). The terms "USB Transfer" and "transfer" are used interchangeably per USB 2.0 Specification.

3. Overview

Open Boot and Management Framework Interface Consolidation Protocol is defined by OCP to consolidate boot and management interfaces to platform components. This document defines USB Device Class Specification for native support of OBMF-ICP over USB. OBMF-ICP over USB SHALL implement Bulk IN and Bulk OUT endpoints and MAY implement Interrupt IN and/or Interrupt OUT endpoints for exposing OBMF-ICP channels to the Host. OBMF-ICP USB Interface SHALL use class-interface descriptor and associated functional descriptor embedded within the device's normal run-time descriptors to expose class-specific interface.

OBMF-ICP over USB MUST implement mandatory Bulk IN and Bulk OUT endpoints for channel discovery, channel access and configuration of OBMF-ICP channels.

OBMF-ICP over USB MAY implement OPTIONAL Interrupt IN and Interrupt OUT for OBMF-ICP channels that require specific quality of service. This type of channel e.g. guaranteed GPIO or Virtual Wire, MAY be added to OBMF-ICP Specification.

4. OCP OBMF-ICP USB Interface definition

OBMF-ICP Interface use standard USB device enumeration process including device descriptor, configuration descriptor, and interface descriptor exchange compatible with USB 1.1 and later host controllers.
OBMF-ICP over USB MUST use Miscellaneous Class bInterfaceClass = 0xEF with a unique bInterfaceSubClass = 0x09 value assigned by USB-IF to uniquely identify OCP OBMF-ICP interfaces across all USB versions and distinguish them from other device functions see https://www.usb.org/defined-class-codes.

4.1 USB Interface Descriptor

Offset	Field	Size	Value	Description
0	bLength	1	0x09	Size of this descriptor in bytes.
1	bDescriptorType	1	0x04	INTERFACE descriptor type.
2	bInterfaceNumber	1	Number	Number of this interface.
3	bAlternateSetting	1	0x00	Alternate setting. Must be zero.
4	bNumEndpoints	1	0x02, 0x03 or 0x04	Bulk IN, Bulk OUT are mandatory Interrupt IN, Interrupt OUT support is OPTIONAL based on device OBMF-ICP capability - 0x02: Bulk IN/OUT (minimum) - 0x03: (Bulk IN/OUT + Interrupt IN) or (Bulk IN/OUT + Interrupt OUT) *) - 0x04: Bulk IN/OUT + Interrupt IN/OUT *) Interrupt direction per endpoint descriptor
5	bInterfaceClass	1	0xEF	Miscellaneous Class Code. Used to bind the driver
6	bInterfaceSubClass	1	0x09	OCP OBMF-ICP InterfaceSubClass 09h. Used to bind the OBMF driver (refers to this document and future OBMF protocols)
7	bInterfaceProtocol	1	0x01	OCP OBMF-ICP USB Device Class Specification version 1.x as assigned by usb.org. Used to bind the driver that supports the current USB OBMF-ICP Device Class Specification 1.x (bInterfaceProtocol=1 refers to this document)
8	iInterface	1	Index	Index of string descriptor for this interface pointing to "OCP OBMF" (8 characters with standard USB encoding bLength and UTF-16LE encoded)

4.2 OBMF-ICP Class-Specific Functional Descriptor

Offset	Field	Size	Value	Description
0	bLength	1	0x0E	Size of this descriptor in bytes.
1	bDescriptorType	1	0x24	Interface-specific/Class-specific descriptor CS_INTERFACE
2	bDescriptorSubtype	1	0x01	Interface-specific subtype for OCP OBMF-ICP OCP_OBMF_FUNCTIONAL
3	bMultimessageSupport	1	Value	0x00 – Device supports only single OBMF-ICP Message per-single USB Transfer. Single OBMF-ICP Message may span across multiple Bulk packets up to wMaxWrTransferSize or wMaxRdTransferSize depending on the direction. For Interrupt endpoints single OBMF-ICP Message must fit into single transfer. 0x01..0xFF – Reserved
4	wMaxWrTransferSize	2	Value	Maximum write transfer size in bytes for the Bulk OUT endpoint
6	wMaxRdTransferSize	2	Value	Maximum read transfer size in bytes for the Bulk IN endpoint
8	wMaxWrInterruptSize	2	Value	Maximum write interrupt EP size in bytes. SHALL fit into single Interrupt OUT transfer size. SHALL be set to 0 bytes if interrupt OUT endpoint is not supported by the OBMF-ICP on this Device
10	wMaxRdInterruptSize	2	Value	Maximum read interrupt EP size in bytes. SHALL fit into single Interrupt IN transfer size. SHALL be set to 0 bytes if interrupt IN endpoint is not supported by OBMF-ICP on this Device
12	bcdOCPOBMFVersion	2	BCD	Numeric expression identifying the Major and Minor version of the OCP OBMF-ICP Specification release implemented by this Device (bcdOCPOBMFVersion MUST NOT refer to this

				document version but to the OBMF-ICP Specification).

4.3 USB Interface Endpoint Descriptors

4.3.1 USB Interface Endpoint Descriptor for Bulk OUT (Mandatory)

Offset	Field	Size	Value	Description
0	bLength	1	0x07	Size of this descriptor in bytes
1	bDescriptorType	1	0x05	ENDPOINT descriptor type
2	bEndpointAddress	1	Value	OUT endpoint address (Bit 7=0: OUT; Bits[3:0]=endpoint number)
3	bmAttributes	1	0x02	Bulk transfer type
4	wMaxPacketSize	2	Value	Max packet size (e.g., 0x0200 = 512 for HS)
6	bInterval	1	0x00	Ignored for Bulk endpoints

4.3.2 USB Interface Endpoint Descriptor for Bulk IN (Mandatory)

Offset	Field	Size	Value	Description
0	bLength	1	0x07	Size of this descriptor in bytes
1	bDescriptorType	1	0x05	ENDPOINT descriptor type
2	bEndpointAddress	1	Value	IN endpoint address (Bit 7=1: IN; Bits[3:0]=endpoint number)
3	bmAttributes	1	0x02	Bulk transfer type
4	wMaxPacketSize	2	Value	Max packet size (e.g., 0x0200 = 512 for HS)
6	bInterval	1	0x00	Ignored for Bulk endpoints

4.3.3 USB Interface Endpoint Descriptor for Interrupt OUT (Optional)

Offset	Field	Size	Value	Description
0	bLength	1	0x07	Size of this descriptor in bytes
1	bDescriptorType	1	0x05	ENDPOINT descriptor type
2	bEndpointAddress	1	Value	OUT endpoint address (Bit 7=0: OUT; Bits[3:0]=endpoint number)

3	bmAttributes	1	0x03	Interrupt transfer type
4	wMaxPacketSize	2	Value	Max packet size. Recommended size ≤64 bytes to minimize the impact on the USB bandwidth allocation.
6	bInterval	1	Value	Polling interval (1-255 msec for FS; 2^(bInterval-1)×125µsec with bInterval value MUST be between 1 and 16 for HS)

Support for Interrupt OUT is optional and shall be added only if there is a specific OBMF-ICP channel that supports Interrupt OUT e.g. guaranteed GPIO or Virtual Wire.

4.3.4 USB Interface Endpoint Descriptor for Interrupt IN (Optional)

Offset	Field	Size	Value	Description
0	bLength	1	0x07	Size of this descriptor in bytes
1	bDescriptorType	1	0x05	ENDPOINT descriptor type
2	bEndpointAddress	1	Value	IN endpoint address (Bit 7=1: IN; Bits[3:0]=endpoint number)
3	bmAttributes	1	0x03	Interrupt transfer type
4	wMaxPacketSize	2	Value	Max packet size. Recommended size ≤64 bytes to minimize the impact on the USB bandwidth allocation.
6	bInterval	1	Value	Polling interval (1-255 msec for FS; 2^(bInterval-1)×125µsec with bInterval value MUST be between 1 and 16 for HS)

Support for Interrupt IN is optional and shall be added only if there is a specific OBMF-ICP channel that supports Interrupt IN e.g. guaranteed GPIO or Virtual Wire.

4.4 USB Descriptors for OCP OBMF-ICP Interface

A USB device implementing OCP OBMF-ICP SHALL expose a minimum set of descriptors for runtime operations, including:

- A Device Descriptor with bDeviceClass=0, bDeviceSubClass=0, and bDeviceProtocol=0 indicating a USB Composite Device where individual interfaces define their respective classes;
- At least one Configuration Descriptor defining the device's operational configuration;
- Exactly one Interface Descriptor dedicated to the OCP OBMF-ICP Interface with bInterfaceClass=0xEF (Miscellaneous), bInterfaceSubClass=0x09 (OBMF-ICP) and bInterfaceProtocol=0x01 (OBMF-ICP current USB Device Class Specification 1.x).
- Exactly one OCP_OBMF_FUNCTIONAL descriptor specifying OBMF-ICP capabilities and transfer limitations.
- Two, Three or Four endpoint descriptors.

5. OCP OBMF-ICP over USB Device Class

The OCP OBMF-ICP interface is dedicated solely to OBMF-ICP communication. There is no additional encapsulation for Bulk OUT, Bulk IN, and OPTIONAL Interrupt OUT, Interrupt IN endpoints. The Interrupt IN and Interrupt OUT endpoints are designated only to pass OBMF-ICP Messages as a notification with data e.g. generating GPIO state change notifications or Virtual Wire, if supported by the device. The channel that allows the use of Interrupt IN or Interrupt OUT MUST be defined in OBMF-ICP Specification.

OBMF-ICP control traffic is using channel 0 over Bulk IN and Bulk OUT endpoints. The size of the endpoints is defined by the device.

Implementation Note: *USB host is expected to continuously attempt reading on Bulk IN to guarantee a minimum latency. It is recommended to use USB Controller to schedule Bulk IN transfer to minimize latency. USB Device will respond with NAK if no data is available on any OBMF-ICP channel.*

OBMF-ICP Bulk IN, Bulk OUT allows for using multiple channels concurrently. The ordering is defined within channel. In current OBMF-ICP Specification, only one outstanding OBMF-ICP Message that requires response is allowed.

OBMF-ICP Messages MAY span multiple USB packets using standard multi-packet transfers on Bulk IN/OUT endpoints.

OBMF-ICP Messages transmitted via Interrupt IN or Interrupt OUT endpoints SHALL fit within a single USB transfer. Multi-packet spanning is NOT SUPPORTED on Interrupt endpoints.

5.1 Length of the OBMF-ICP Message that spans across multiple packets:

1. Bulk OUT transfer ends when the Host sends a short packet (shorter than wMaxPacketSize). If the payload is an exact multiple of wMaxPacketSize, the Host MUST send a ZLP to terminate the current OBMF-ICP payload. It is expected the Device will check the USB transfer size with the OBMF-ICP channel encapsulation.
2. Bulk IN transfer ends when the Host reads from Device a short packet (shorter than wMaxPacketSize). If the payload is an exact multiple of wMaxPacketSize, the device MUST send a ZLP to terminate the current OBMF-ICP payload. It is expected the Host will check the transfer size with the OBMF-ICP Message size.
3. Interrupt OUT transfer ends on each service interval the host sends for up to wMaxWrInterruptSize bytes. If host has no data, it will not initiate any transfer and will skip the service interval. It is expected the Device will check the transfer size with the payload size as specified by OBMF-ICP header.

4. Interrupt IN transfer ends on each service interval the host asks for up to wMaxRdInterruptSize bytes. If device has no data, it NAKs. It is expected the Host will check the transfer size with the payload size as specified by OBMF-ICP header.

6. Compatibility

This OBMF-ICP USB Device Class Specification is compatible with USB 1.1 and higher.

7. Appendix: USB Interface Implementation notes (*Informative only*)

7.1 USB Transfer Error Handling

OBMF-ICP over USB uses USB transfer integrity checking through standard USB CRC mechanisms and packet validation as defined in the respective USB specification versions (1.1, or later). There is no additional integrity needed, unless enforced by the OBMF-ICP specification. If a USB transfer is interrupted or incomplete as defined by the USB specification USB endpoint SHALL silently discard this OBMF-ICP Message. Subsequent transfers SHALL start with new OBMF-ICP Message after recovery of OCP OBMF-ICP USB Interface. Failed transfer SHALL be detected immediately based on STALL condition.

7.2 USB Interface Error Recovery

USB Device supporting OBMF-ICP over USB SHALL implement CLEAR_FEATURE(ENDPOINT_HALT) and BUS Reset sensing. Those events MAY result in OBMF-ICP protocol recovery see implementation note below.

USB Host will issue CLEAR_FEATURE(ENDPOINT_HALT) and USB PORT RESET or BUS RESET to recover OBMF-ICP over USB. Those actions are typically executed automatically by the USB Host stack.

OBMF-ICP over USB Interface recovery employs standard USB error recovery mechanisms to handle STALL conditions as defined by USB specification. Comprehensive USB-specific error recovery procedures are defined to ensure high resilience in configurations where USB serves as the primary legacy interface aggregation protocol. The following hierarchical escalation approach shall be implemented when OBMF-ICP channel operations fails, progressing from least to most disruptive recovery methods:

1. Attempt CLEAR_FEATURE(ENDPOINT_HALT): On clearing STALL, OBMF-ICP USB Interface implementation on Device employs sufficient methods to ensure subsequent OBMF-ICP Channel Transfers are handled as expected without the need for further intervention. This includes, but is not limited to, flushing internal pipelines and resetting counters when a STALL condition is cleared by issuing CLEAR_FEATURE(ENDPOINT_HALT) on OBMF-ICP USB Interface.
2. Attempt a USB PORT RESET or BUS RESET by driving both D+ and D- lines to low state (SE0) for the minimum duration specified in the USB specification (e.g., 10ms for typical USB Device implementations). The reset shall be initiated by either a USB Hub (for port reset via SET_PORT_FEATURE) or USB Host Controller (for bus reset) depending on the USB physical topology. The USB reset signaling is expected to be detected by dedicated USB Device hardware (e.g., USB PHY for USB 2.0 High Speed and higher). USB reset detection shall be isolated from the USB receive/transmit logic to ensure reliable reset recognition regardless of the current USB communication state. Each of the above methods is expected to bring the OBMF-ICP USB Interface function on the USB Device to an operational state.

The underlying mechanism for each of the attempts is implementation specific.

8. About Open Compute Foundation

The Open Compute Project (OCP) brings at-scale innovations and hyperscaler best practices to all, spanning technology domains from the data center to the edge, and the technology stack from silicon, to systems, to site facilities and services. The international OCP Community is made up of organizations and people from hyperscale and tier-2 cloud data center operators, communications providers, colocation providers, diverse enterprises, and technology vendors. The OCP Foundation fosters collaboration within the Community to tackle market challenges in an array of OCP Projects that create open contributions such as specifications, designs, and more. The OCP Solution Provider Program then recognizes the application of those contributions in compliant products, solutions, and data center facilities and services with designations like OCP Inspired, OCP Accepted and OCP Ready. With the tenets of openness, impact, efficiency, scale and sustainability, the OCP engages and educates thousands of engineers every year through many events and webinars. Across many initiatives the OCP Foundation and Community are meeting the market today and shaping the future.

Learn more at: www.opencompute.org.